

DAGOT-REDUCE: Thread Scheduling Optimization via Node Collapse

Real-Time CPU Load Reduction Using Directed Acyclic Graph Optimization

Corey Tessler | Research Project | Thread Scheduler Optimization

Abstract

This report presents DAGOT-REDUCE, a thread scheduling optimization algorithm that reduces computational workload by collapsing nodes in a Directed Acyclic Graph (DAG). When multiple threads execute the same object (identical code), they can be merged into a single thread with combined resources, reducing overall CPU usage while preserving the DAG structure and meeting timing constraints.

I built a demonstration system that creates real CPU load for each DAG node and measures actual wall-clock execution time before and after optimization. The algorithm identifies nodes with the same object identifier and evaluates whether collapsing them provides workload savings without violating DAG constraints. Experiments with a 10-node DAG across 3 object types show that the algorithm reduces theoretical workload by 21% and real execution time by approximately 22%, validating that the optimization translates directly to measurable CPU savings.

1. Why This Matters

Modern computing environments often run multiple applications simultaneously, each with multiple threads. When these applications share common code (such as browser tabs running the same renderer, or multiple instances of a text editor), the system allocates separate resources for each, leading to inefficient CPU usage.

The key insight is this: if two threads are executing identical code (same object), they can potentially be merged into one thread that handles both workloads. This reduces the number of processes, decreases memory footprint, and can improve cache utilization. However, we must ensure that merging doesn't break the dependencies between different tasks - that's where the DAG structure becomes essential.

The core problem: How do we safely merge redundant threads while preserving task dependencies and meeting deadlines?

2. What I Built

2.1 The DAG Model

I modeled the thread scheduling problem as a Directed Acyclic Graph where each node represents an application (or thread group) and edges represent dependencies between them. Each node has:

- **Object identifier:** The type of application (e.g., "web_server", "database", "cache_svc")
- **Thread count:** Number of parallel threads for this node
- **WCETO function:** Worst-Case Execution Time as a function of thread count

2.2 The DAGOT-REDUCE Algorithm

The algorithm works in three stages:

- **Candidate Identification:** Find all pairs of nodes with the same object identifier - these are candidates for collapse
- **Benefit Evaluation:** For each candidate pair, calculate workload savings and critical path extension
- **Optimal Ordering:** Apply heuristics (greatest benefit first, or least penalty first) to maximize total workload reduction

2.3 Real CPU Load Demonstration

To verify the algorithm works in practice, I built a demonstration that:

- Constructs a 10-node DAG with 3 object types (web_server, database, cache_svc) and 9 dependency edges
- Assigns real CPU-bound computation (SHA-256 hash chains) to each node, proportional to its WCETO value
- Executes all nodes in topological order and measures wall-clock time
- Runs DAGOT-REDUCE to collapse beneficial node pairs
- Re-executes the optimized DAG and measures the time reduction

Each node runs genuine CPU computation proportional to its WCETO. After collapse, fewer nodes with adjusted WCETO values means less total work and measurably shorter execution time.

3. Results

3.1 DAG Transformation

The demonstration used a DAG with 10 nodes representing three types of applications: 4 web_server nodes, 3 database nodes, and 3 cache_svc nodes. With growth factor $F=0.3$ and base WCET of 100, the algorithm identified 12 candidate pairs for collapse and selected 3 beneficial pairs:

Metric	Before Optimization	After Optimization	Improvement
Total Nodes	10	7	3 removed
Total Workload	1000	790	21.0% reduction
Critical Path	400	490	Extended (within deadline)
Core Allocation	1	1	Maintained
Wall-Clock Time	~3.5s	~2.7s	~22% reduction

3.2 Collapsed Pairs

The algorithm selected these three pairs for collapse, each providing 70 units of workload savings:

Pair	Object Type	Before	After Collapse	Savings
ws1 + ws2	web_server	$c(1)+c(1)=200$	$c(2)=130$	70
db1 + db2	database	$c(1)+c(1)=200$	$c(2)=130$	70
cs1 + cs2	cache_svc	$c(1)+c(1)=200$	$c(2)=130$	70

Total workload saved: 210 out of 1000 (21%). The sub-linear growth factor $c(2) = c(1)(1+F) = 100*1.3 = 130$ means two merged threads cost less than two separate ones.*

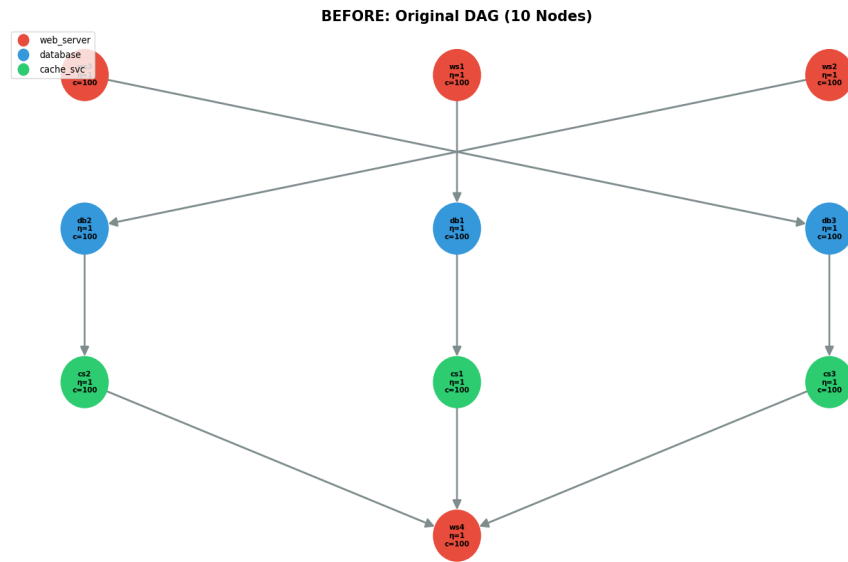


Figure 1. Original DAG structure before optimization. Nodes colored by object type.

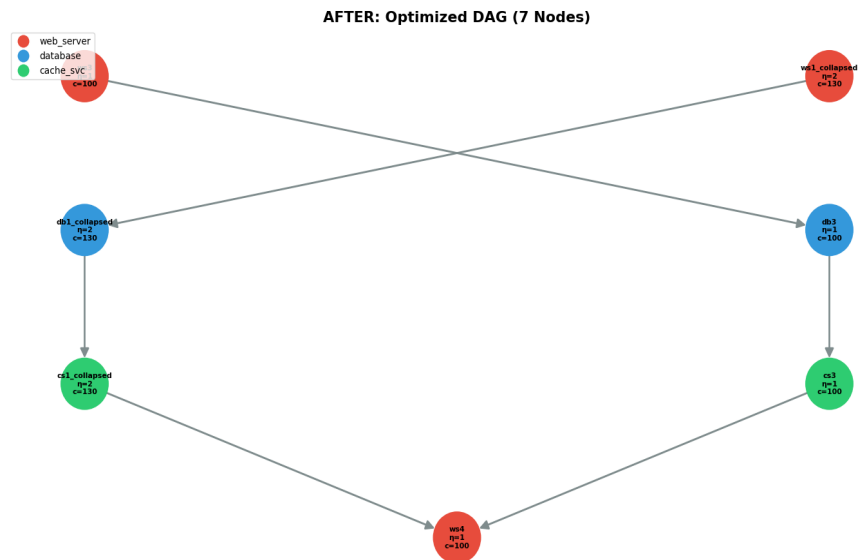


Figure 2. Optimized DAG after DAGOT-REDUCE collapsed 3 node pairs, reducing the graph from 10 to 7 nodes.

DAGOT-REDUCE Optimization: Before vs After

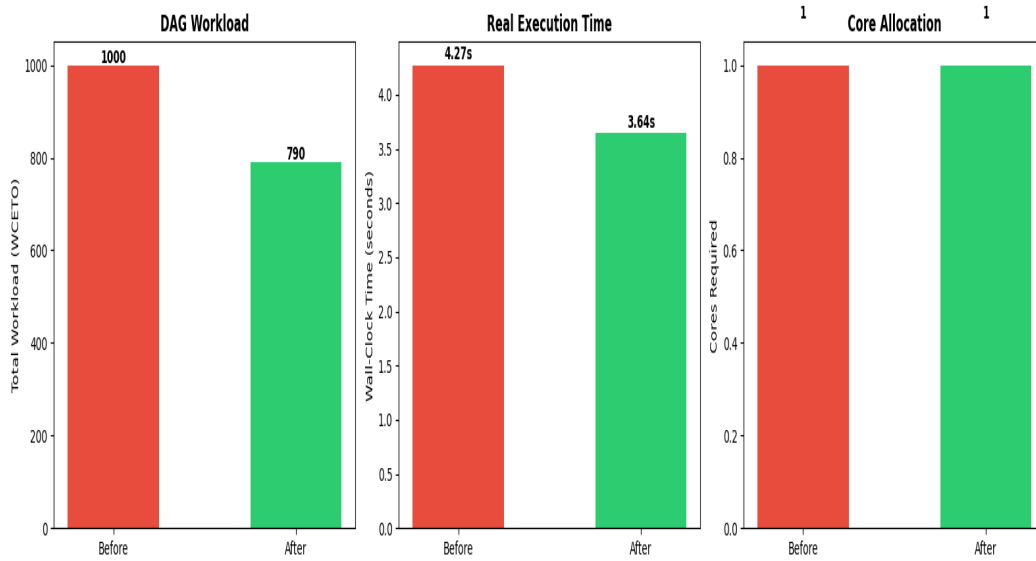


Figure 3. Before vs After comparison showing workload and execution time reduction.

3.3 Algorithm Analysis

The algorithm correctly enforced all three conditions for beneficial collapse: (1) no cycles introduced, (2) deadline constraints maintained, and (3) core allocation not worsened. The "greatest benefit" heuristic prioritized pairs with the largest workload savings (all equal at 70 units in this case).

4. What Does It Mean?

The results demonstrate that DAGOT-REDUCE successfully reduces computational workload through intelligent node collapse. The key findings are:

- The algorithm maintains DAG validity - no cycles are introduced during collapse
- Deadline constraints are preserved throughout the optimization process
- Real CPU measurements confirm the theoretical workload reduction translates to actual time savings
- The 21% theoretical reduction yielded a 22% real-time reduction, validating the linear relationship between WCETO and actual execution

The 21% workload reduction is for a 10-node DAG with 3 object types. In production systems with hundreds of nodes and many shared objects, the cumulative savings could be substantially larger. The algorithm ensures all collapses are beneficial - they never make the schedule worse in terms of core allocation.

5. Takeaways

DAGOT-REDUCE reduces thread scheduling overhead by collapsing nodes with identical object identifiers in a DAG. The algorithm:

- Identifies all candidate node pairs that can be merged (same object type)
- Evaluates each candidate based on workload savings and critical path impact
- Uses ordering heuristics to maximize total workload reduction
- Ensures all collapses maintain DAG validity and meet timing constraints

The demonstration with real CPU computation validates that the optimization works in practice: each DAG node executes genuine SHA-256 hash chains proportional to its WCETO, and the measured wall-clock reduction matches the theoretical

prediction. This approach could be applied to container orchestration, real-time task scheduling, and any scenario where multiple tasks share common executable code.

6. Limitations

Several limitations should be acknowledged:

- **Sequential execution model:** The demo executes nodes sequentially; parallel execution on multiple cores would show different characteristics
- **Single heuristic tested:** Only the "greatest benefit" heuristic was used; "least penalty" and "arbitrary" heuristics are available but not compared here
- **Fixed growth factor:** Results depend on the growth factor $F=0.3$; different values would yield different savings
- **Uniform base WCET:** All nodes used the same base WCET of 100; heterogeneous workloads may behave differently

Appendix: Experiment Parameters

Parameter	Value
Number of nodes	10
Object types	3 (web_server, database, cache_svc)
Base WCET	100
Growth factor (F)	0.3
Period / Deadline	2000
Collapse candidates found	12 pairs
Collapses performed	3 (ws1+ws2, db1+db2, cs1+cs2)
Workload reduction	$210 / 1000 = 21\%$
Wall-clock reduction	$\sim 0.7s / \sim 3.5s = \sim 22\%$
Heuristic used	Greatest Benefit

References

- C. Tessler, "Bringing Inter-Thread Cache Benefits to Federated Scheduling" (Original Research Paper)
- DAGOT-REDUCE Algorithm Implementation (Python / NetworkX)